Computer Vision - Spring 26

Assignment 4

Instructors: Prof Makarand Tapaswi and Prof Charu Sharma

Deadline: 19 April 2026 11:59 P.M.

General Instructions

- Accept the assignment using this link : `https://classroom.github.com/a/QI2enmXr`
- Your assignment must be implemented in Python.
- Plagiarism will only be taken into consideration for code that is not generated by LLM services. Any code generated with the assistance of LLM services should be considered as a resource, similar to using a textbook or online tutorial.
- You are responsible for every line of code in your codebase.
- Clearly label and organize your code, including comments that explain the purpose of each section and key steps in your implementation.
- Properly document your code and include explanations for any non-trivial algorithms or techniques you employ.
- Ensure that your files are well-structured, with headings, subheadings, and explanations as necessary. Your assignment will be evaluated not only based on correctness but also on the quality of code, the clarity of explanations, and the extent to which you've understood and applied the concepts covered in the course.
- Make sure to test your code thoroughly before submission to avoid any runtime errors or unexpected behavior.
- Report all the analysis, comparison and any metrics in the notebook or a separate report that is part of the submission itself. No external links to cloud storage files, wandb logs or any other alternate will be accepted as part of your submission. Only the values and visualizations as part of your commits will be graded.
- Save your visualizations in a separate folder, or display them in the notebook in a lower resolution, in case size of the file is huge.

1 ViT and variations [50 Marks]

1.1 Standard ViT Encoder for CIFAR-10 Classification

In this task, you will implement and train the Vision Transformer (ViT) from scratch on the CIFAR-10 dataset. You can load this from *torchvision* library.
Reference paper: `https://arxiv.org/abs/2010.11929`

1.1.1 Tasks

- Implement scaled dot product attention and multi-headed attention blocks as described in the ViT paper.(Do not use inbuilt functions like nn.MHA or nn.TransformerEncoder)
- Use these, and incorporate them in the transformer encoder layer. Add a [CLS] token.
- Add learnable positional embedding as implemented in the paper.
- You can use patch size = 4 for your initial experiments.

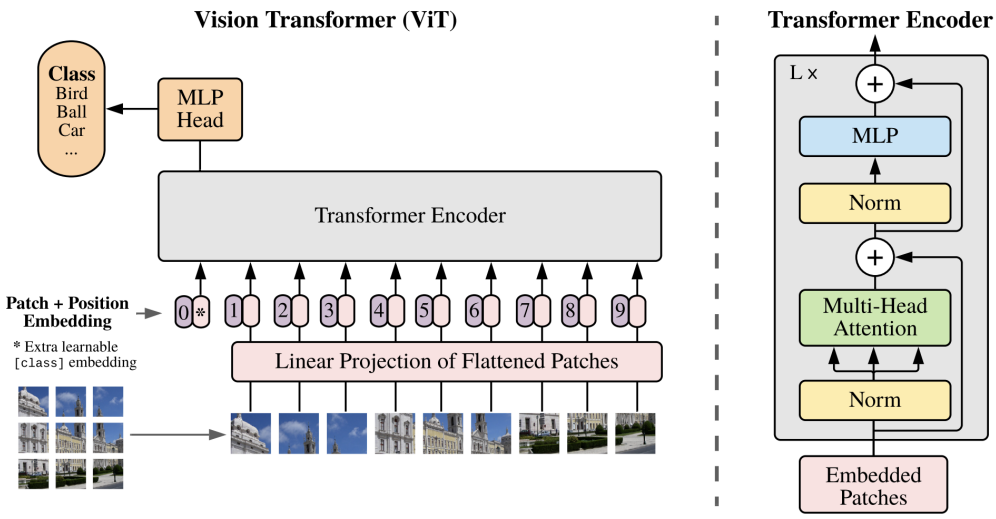

Figure 1: ViT architecture

- Train the ViT model on CIFAR-10 (Do not resize), plot the loss curves, and report the test accuracy. Make sure to think through weight initializations and learning rate schedulers beforehand.
- Consider using random seed for reproducibility for your own convenience.

1.1.2 Some interesting concepts to read

1. PreNorm vs PostNorm
   - The original Transformer uses PostNorm whereas later models inluding ViT use PreNorm, the difference is quite sutble but its training implications are interesting to read.
     Reference : https://arxiv.org/abs/2002.04745
   - Try to find the variance of outputs of MHA and MLP after each layer in both these cases. Try to derive an expression using definitions of expectation and variance(It would be fun to work it out rather than asking Claude).
   - Based on the results, what do you think will happen by stacking too many layers?
     Reference : https://arxiv.org/pdf/2502.05795
2. LayerNorm vs BatchNorm
   - Unlike CNN family where BatchNorm is widely used, Transformers adopt LayerNorm.
     Reference : Section-2 of `https://proceedings.mlr.press/v119/shen20e/shen20e.pdf`

1.1.3 Experiments

1. **Patch Size Variation:**
   - Train and evaluate the model with patch sizes of 2, 4 (default), and 8.
   - Plot the loss curves and report the test accuracies.
2. **Hyperparameter exploration:**
   - Experiment with hyperparameters such as embedding dimension, number of transformer layers, MLP hidden dimension, and number of attention heads etc. Identify the best hyperparameter combination that achieves more than 80% test accuracy within 50 epochs. It is a good practice to use early stopping since some models converge quite quickly.
   - Report your best configuration and discuss your findings.

3. **Data Augmentations:**

   - Explore the effect of different data augmentation techniques on model performance from *torchvision.transforms*. Report the best test accuracy achieved and specify the augmentation techniques (and combinations) used. You can try three different augmentation techniques or a combination thereof to improve the performance further.

   Note : You are not expected to do an exhaustive hyperparameter training with all permutations. However, you are expected to understand how each hyperparameter effects the model based on the context of the given task.

1.1.4 Positional Embeddings

Using the best hyperparameters identified, implement the following types of positional embeddings (refer to Appendix D.4 in the ViT paper):

- No positional embedding
- 1D learned (default and used in previous experiments)
- 2D learned positional embedding (Refer point 3 of Appendix D.4)
- Classic sinusoidal positional embedding from the original Transformer paper (Attention Is All You Need).

Report and compare the loss curves and test accuracy for each positional embedding.

1.1.5 Visualizations

1. **DINO Attention Maps:**

   - Visualising the attention map from [CLS] token to patch tokens was popularized in DINO. This approach highlighted ViT's capability to capture the semantic layout of an image through their attention maps. In this section, you will be running the DINO's code to visualize the attention map of a pre-trained ViT given by DINO.
   - Use the following implementation from this repository and run `visualize_attention.py` using any two images of your own choosing and visualize the attention maps.

2. **VIT CIFAR-10 Attention Maps:**

   - Visualize the attention maps of the ViT model trained on CIFAR-10 in the previous experiments.
   - Specifically, visualize the attention from the [CLS] token to all patch tokens for each attention head of the last attention layer, as well as the aggregated attention map (e.g., by averaging across heads).
   - Additionally, visualize the attention from [CLS] to all patch tokens for all the attention layers in the model.

3. **Attention Rollout:**

   - To compute the attention maps from the output token to the input space, ViT paper used attention rollout. Refer to Appendix D.8 of the ViT paper for details.
   - Attention Rollout was introduced in the paper Quantifying Attention Flow in Transformers. You can refer to Section 3 (i.e. the attention rollout sections only) of the paper below. Reference Paper: `https://arxiv.org/abs/2005.00928`
   - In this task, you have to implement the attention rollout as implemented in the ViT paper and visualize the attention maps. You can use images from the CIFAR-10 test set for visualization.

4. **Positional Embedding Visualization:**

   - Use the trained ViT with default learned 1D positional embeddings from the initial task. Visualize the similarities between the learned positional embedding with itself by taking their dot-product.

1.2 Convolutional ViT

In this task, you will extend the standard Vision Transformer architecture by implementing the Convolutional vision Transformer (CvT). This architecture introduces convolutions to merge the desirable properties of CNNs (local receptive fields, spatial subsampling) with the attention of Transformers.

The primary objective is to understand how the inductive bias of CNNs can be integrated into a Transformer framework, where Multi-Head Attention typically operates at a global level. You will explore how replacing linear projections with convolutional ones affects the model's ability to learn spatial structures.

Reference Paper: `https://arxiv.org/abs/2103.15808`

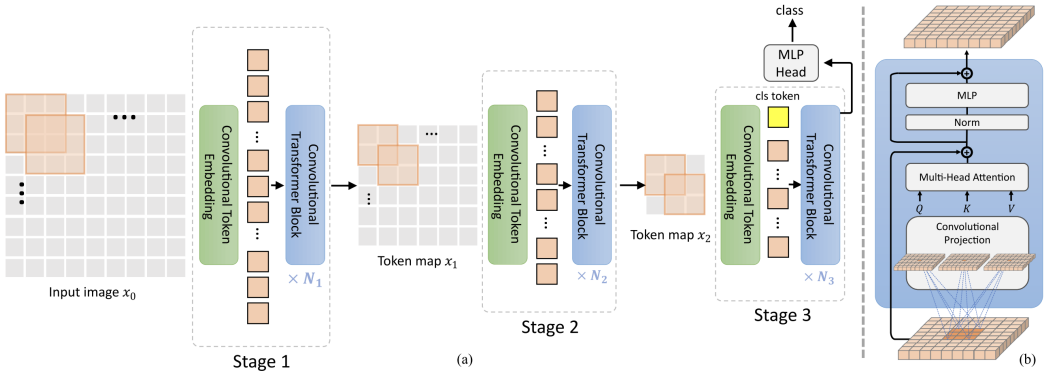

Figure 2: Convolutional ViT architecture

1.2.1 Tasks

- Implement the **Convolutional Token Embedding** module. This should use overlapping 2D convolutions (with stride and padding) to progressively reduce sequence length and increase feature dimension across stages, completely replacing the standard non-overlapping patch embedding.
- Implement the **Convolutional Projection** module. Replace the standard position-wise linear projections for Query, Key, and Value matrices with depth-wise separable convolutions. Implement a stride > 1 (e.g., stride of 2) for Key and Value projections to reduce computational cost.
- Integrate these modules into a multi-stage hierarchy (e.g., 3 stages) as described in the paper. Note that the [CLS] token should only be appended at the beginning of the final stage.
- **Hint for CIFAR-10:** The original paper's baseline architecture is scaled for 224×224 ImageNet images. If you apply an initial stage stride of 4 to a 32×32 CIFAR image, your spatial dimensions will collapse to 1×1 before the final stage, crippling the self-attention mechanism. Adjust your parameters accordingly.
- Train your CvT model from scratch on CIFAR-10, plot the loss curves, and report the test accuracy.

1.2.2 Experiments

1. **Standard ViT vs. CvT:** Compare the final test accuracy, convergence speed (loss curves), and number of parameters between your best vanilla ViT model (from Section 1.1) and your CvT model. Discuss the differences.
2. **Ablation Study - Positional Embeddings:** The CvT paper claims positional embeddings can be completely removed because convolutions inherently model local spatial relationships. Train a variant of your CvT with the 1D learned positional embeddings added back in. Compare the performance and discuss if the paper's claim holds true for your CIFAR-10 implementation.
3. **Ablation Study - Convolutional Projection:** Train a variant where the Convolutional Projections are reverted to standard linear projections (but retain the multi-stage Convolutional Token Embeddings). Report the impact on test accuracy and discuss the trade-offs.

1.2.3 Visualizations

- Visualize the attention maps from the [CLS] token to all spatial tokens for each attention head in the final attention layer of your trained CvT model.
- *Additional consideration:* Since the tokens have undergone downsampling across the multiple stages, ensure your visualization correctly maps the final stage's tokens back to the corresponding original image regions.

1.3 Expected Directory Structure

```
q1/
├── Report.pdf ..... Or main notebook with explanations.
├── code/ .. All Python scripts or notebooks, try to write modular code, with separate files for each
│   architecture and a driver file importing from these
├── training_curves/ ..... Loss curves and metric plots
│   ├── 1.1_ViT/
│   │   ├── patch_size_variation/ ..... Sec 1.1.3 - Exp 1
│   │   ├── hyperparameter_tuning/ ..... Sec 1.1.3 - Exp 2
│   │   ├── data_augmentations/ ..... Sec 1.1.3 - Exp 3
│   │   └── positional_embeddings/ ..... Sec 1.1.4
│   └── 1.2_CvT/
│       ├── standard_vs_cvt/ ..... Sec 1.2.2 - Exp 1
│       ├── ablation_pos_embed/ ..... Sec 1.2.2 - Exp 2
│       └── ablation_conv_proj/ ..... Sec 1.2.2 - Exp 3
└── visualizations/ ..... Saved images and heatmaps
    ├── 1.1_ViT/ ..... Sec 1.1.5
    │   ├── dino_attention_maps/
    │   ├── vit_cifar10_maps/
    │   ├── attention_rollout/
    │   └── pos_embed_similarity/
    └── 1.2_CvT/ ..... Sec 1.2.3
        └── cvt_attention_maps/
```

Note: The subfolders in ViT and CvT are not rigid, you can create folders based on your hyperparameter tuning.
The purpose of the directory structure is not to put additional burden but for ease of evaluation beneficial to both yourself and TAs. Small deviations are still fine but make sure to include these details at the beginning of the report.

2 CLIP [20 Marks]

2.1 Tasks

1. **Inference using CLIP.** Load the ResNet-50 (RN50) model, initialized in two different ways:

   - ImageNet pretraining (`torchvision.models`, specifically `IMAGENET1K_V1`); and
   - OpenAI's CLIP.

   Do the visual encoders have the same architecture? If not, describe and explain the differences.

   Hint: When loading the CLIP model, you will get both the vision and text encoders—be sure to differentiate between them.

2. **Setup data.** Understand the ImageNet challenge dataset (1000 labels of ILSVRC).

   - What label hierarchy is used in ImageNet?
   - What does a synset mean?
   - Could grouping objects based on synsets lead to problems for visual recognition?
   - State three types of visual differences we can expect to see in images with objects corresponding to the same synset.

3. **Setup zero-shot CLIP.** Similar to the ImageNet pretrained RN50, set up CLIP to generate probability scores for the 1000 ImageNet categories.

   Test it with a few example images to verify that it identifies the correct object category.

   Hint: You may treat cosine similarities as "logits".

4. **CLIP vs ImageNet pretraining.** Pick 10 classes from ImageNet (not all from the same branch, e.g., not all dogs). For each class:

   - Find 2 images that work well with CLIP, but not with ImageNet pretrained RN50. Explain why this may be the case. Mention the source of these images.
   - Find 1 image that works well with ImageNet pretraining but not CLIP. Explain why this may be the case. Mention the source of these images.

   Note: A model "works well" if the correct category appears in the top-5 predictions.

   Hint: Refer to the CLIP paper (`https://arxiv.org/abs/2103.00020`) for examples.

3. ClipCap: CLIP Prefix for Image Captioning [50 Marks]

Reference Paper: https://arxiv.org/abs/2111.09734
Reference Repository: https://github.com/rmokady/CLIP_prefix_caption
Clone the repository and use the provided pretrained checkpoints throughout this question. No training from scratch is required.

3.1 Fine-tuning Ablation [15 Marks]

The paper proposes two model variants: (a) a frozen GPT-2 with a Transformer mapper, and (b) a fine-tuned GPT-2 with an MLP mapper. Using the pretrained checkpoints provided in the repository:

- Load both pretrained variants — `MLP + GPT-2 tuned` and `Transformer (frozen GPT-2)` — and run inference on a random subset of **1000 images** from the COCO Captions 2014 val set.
- **Hint:** Download the COCO 2014 validation images and the Karpathy split annotations from the following link.You only need to run inference on a random 1000 of these.
- Report CIDEr and BLEU@4 for each variant using the official `pycocoevalcap` toolkit.
- Compare both configurations in a single table reporting CIDEr, BLEU@4, and number of trainable parameters.
- Discuss: when does fine-tuning the language model help, and when does it hurt? Connect your observations to the paper's hypothesis about dataset style complexity.

3.2 Prefix Length Analysis [10 Marks]

- Using the best-performing pretrained variant, modify the prefix length k at inference time by truncating or zero-padding the prefix embeddings to lengths $k \in \{1, 5, 10, 20, 40\}$.
- For each k, report CIDEr and BLEU@4 on the COCO test set.
- Plot performance vs. prefix length for both metrics on the same graph and discuss the saturation behavior. Does increasing prefix length always help? Why or why not?
- **Note:** The model was trained with a fixed k. Think carefully about what truncating or padding the prefix means for the language model, and discuss any limitations of this inference-time analysis.

3.3 Prefix Interpretability [10 Marks]

- For both the **MLP + GPT-2 tuned** model and the **frozen Transformer mapper** model, extract the k prefix embeddings for 5 test images of your choice.
- Interpret each prefix token by finding its nearest neighbor in GPT-2's vocabulary embedding matrix under cosine similarity.
- Display the image, generated caption, and the sequence of nearest-neighbor words for each prefix token side by side for both variants.
- Discuss the difference in interpretability between the two variants. Why are the frozen mapper's prefixes largely unreadable, as analyzed in Section 4 of the paper?

3.4 Visualizations & Qualitative Analysis [15 Marks]

1. **Cross-modal Alignment** *[5 Marks]*
   For 10 test images, plot a heatmap of cosine similarities between the CLIP image embedding and each of the k generated prefix token embeddings. Do the prefix tokens cluster around the CLIP embedding or diverge? Does this pattern differ between the MLP and Transformer mapper variants? Discuss your observations.
2. **Failure Case Analysis** *[5 Marks]*
   Identify and display at least 5 failure cases from the COCO test set. Categorize each failure using one of the following labels: wrong object, wrong relationship, hallucination, incomplete description, or wrong count. For each case, hypothesize whether the failure originates from CLIP's visual encoding or GPT-2's language decoding, and justify your reasoning.
3. **Generalization to Out-of-Domain Images** *[5 Marks]*
   Run inference (using the pretrained model, no retraining) on 5 images sourced from the web that are clearly outside the COCO distribution — for example: satellite imagery, medical scans, paintings, or memes. Report the generated captions and discuss how well the model generalizes to these unseen domains. What does this reveal about the role of CLIP as the visual backbone?